
Software Requirements Specification

for

Mini project :- Design and Implementation of an $O(1)$ Time-Complexity Key-Value Storage and Eviction Engine

Version 1.0 approved

**Prepared by
Swastik Shivane
UID:- 2024800118
Batch B3 CSE (TY)**

28/07/2026

Table of Contents

Table of Contents.....	ii
Revision History.....	ii
1. Introduction.....	1
1.1 Purpose.....	1
1.2 Document Conventions.....	1
1.3 Intended Audience and Reading Suggestions.....	1
1.4 Product Scope.....	1
1.5 References.....	1
2. Overall Description.....	2
2.1 Product Perspective.....	2
2.2 Product Functions.....	2
2.3 User Classes and Characteristics.....	2
2.4 Operating Environment.....	2
2.5 Design and Implementation Constraints.....	2
2.6 User Documentation.....	2
2.7 Assumptions and Dependencies.....	3
3. External Interface Requirements.....	3
3.1 User Interfaces.....	3
3.2 Hardware Interfaces.....	3
3.3 Software Interfaces.....	3
3.4 Communications Interfaces.....	3
4. System Features.....	4
4.1 System Feature 1.....	4
4.2 System Feature 2 (and so on).....	4
5. Other Nonfunctional Requirements.....	4
5.1 Performance Requirements.....	4
5.2 Availability Requirements.....	5
5.3 Scalability Requirements.....	5
5.4 Security Requirements.....	5
5.5 Reliability Requirements.....	5
5.6 Maintainability Requirements.....	5
6. Other Requirements.....	5
Appendix A: Glossary.....	5
Appendix B: Analysis Models.....	5
Appendix C: To Be Determined List.....	6

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

This document specifies the software requirements for the "Design and Implementation of an $O(1)$ Time-Complexity Key-Value Storage and Eviction Engine". It describes the core architectural constraints, functional boundaries, and performance benchmarks necessary to build the software module.

1.2 Document Conventions

All high-level performance requirements (such as time-complexity constraints) are implicitly inherited by any detailed programmatic requirements. Priorities are treated uniformly unless explicitly stated otherwise.

1.3 Intended Audience and Reading Suggestions

The primary audience for this document includes systems engineering students, computer science professors evaluating the project, and backend developers. Readers should begin with the overall description and architectural concepts before reviewing the empirical benchmarking and specific API functional requirements.

1.4 Product Scope

This project is a low-overhead, in-memory key-value storage engine engineered to guarantee deterministic $O(1)$ constant time complexity across all core operations. These operations include data insertion, lookup, update, and page eviction. By coupling an associative Hash Map with a custom Doubly Linked List, the software enforces a Least Recently Used (LRU) eviction policy while avoiding standard memory reclamation latency bottlenecks of $O(n)$ or $O(n \log n)$.

1.5 References

"An $O(1)$ algorithm for implementing the LFU cache eviction scheme" by Ketan Shah, Anirban Mitra, Dhruv Matani. arXiv:2110.11602.

"ARC: A Self-Tuning, Low Overhead Replacement Cache" by Nimrod Megiddo, Dharmendra S. Modha. Published at USENIX FAST '03.

2. Overall Description

2.1 Product Perspective

This product is a new, self-contained software-level in-memory key-value store module designed to mimic the core internals of industry-standard caching systems like Redis.

2.2 Product Functions

Implement direct data lookups via key hashing algorithms.

Track the relative recency of element access utilizing a Doubly Linked List.

Enforce a Least Recently Used (LRU) eviction policy securely and predictably.

Execute standardized API methods including `get(key)`, `put(key, value)`, and `evict()`.

Dynamically allocate and free memory on the Heap to prevent leaks during eviction processes.

Execute automated synthetic workloads and generate CSV runtime outputs.

2.3 User Classes and Characteristics

The module is targeted primarily for interaction by systems programs rather than direct end-users. The secondary class involves academic evaluators seeking to validate rigorous time-complexity proofs, empirical execution speeds, and proper low-level memory usage.

2.4 Operating Environment

The system must run on standard consumer-grade computer processors, such as a Ryzen 3 3200U (minimum). It requires adequate primary memory (RAM) to support the dynamic allocation of up to 1,000,000 concurrent nodes during intensive stress testing.

2.5 Design and Implementation Constraints

The system codebase must be written in a systems-level programming language such as C++, Java, or Rust.

Standard compilation must be handled by build tools like CMake or Makefiles.

Multithreading, networking configurations, and external cybersecurity threats are strictly out of scope for this localized memory module.

Secondary storage integration (Disk I/O) is not permitted, as execution occurs entirely in-memory.

2.6 User Documentation

The project deliverables will include a structured academic report containing architectural diagrams (Data Structure Layout), algorithmic flowcharts for core logic paths, empirical benchmarking graphs generated via Python/matplotlib, time-complexity mathematical proofs, and terminal logs displaying zero memory leaks from profiling tools like Valgrind.

2.7 Assumptions and Dependencies

It is assumed that the host machine possesses sufficient RAM capacity to execute the benchmarking scripts up to the highest specified threshold of 1,000,000 requests without triggering OS-level virtual memory swapping, which would invalidate the software $O(1)$ performance measurements.

3. External Interface Requirements

3.1 User Interfaces

The system relies entirely on standard programmatic interfaces (API) to perform cache operations and provides no graphical user interface.

3.2 Hardware Interfaces

The software module interfaces directly with primary memory (RAM) for the dynamic allocation and deallocation of cache nodes. There is strictly no interaction with secondary storage hardware disks.

3.3 Software Interfaces

The application will integrate with automated testing suite scripts built directly into the language environment to execute synthetic load loops and export the operational latencies to CSV format for processing. It must also interact successfully with standard build tools and memory profilers (e.g., GCC/G++, CMake, Valgrind).

3.4 Communications Interfaces

This system acts as a pure in-memory structure with no networking requirements. All TCP socket, external event-loop, or multi-client communication standards are omitted from the design.

4. System Features

4.1 Storage and Retrieval Operations

Description and Priority: High priority. The system processes direct read and write actions to standard keys and values.

Stimulus/Response Sequences: The API calls `put(key, value)` or `get(key)`. The system queries the Hash Map to evaluate hits or misses and dynamically updates standard values.

Functional Requirements:

- The system shall implement an associative Hash Map to handle direct data lookups via key hashing algorithms.
- The module must guarantee deterministic $O(1)$ constant time complexity across all insertion and lookup operations.

4.2 LRU Eviction Engine

Description and Priority: High priority. The system enforces memory limitation constraints by deleting obsolete data.

Stimulus/Response Sequences: When an insertion causes the cache to exceed its pre-configured capacity, the system triggers the `evict()` module, automatically purging the Tail node from the linked structure.

Functional Requirements:

- The system shall implement a custom Doubly Linked List to track the relative recency of element access.
- The software must combine the Hash Map and Doubly Linked List to create a hybrid data structure that enforces a Least Recently Used (LRU) eviction policy.
- The program shall dynamically allocate memory for nodes on the Heap and properly free it to prevent leaks during eviction.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The system must guarantee deterministic $O(1)$ constant time complexity across all core operations, including data insertion, lookup, and page eviction.

Standard memory reclamation latency bottlenecks, such as $O(n)$ or $O(n \log n)$, must be strictly avoided.

5.2 Availability Requirements

The engine must remain highly accessible during intense synthetic workloads without locking up. System uptime is localized to the runtime of the benchmarking scripts and must complete execution loops up to 1,000,000 requests without stalling.

5.3 Security Requirements

Because this module operates locally without networking or multithreading, external cybersecurity threats are out of scope.

Internal security and stability will be maintained by preventing segmentation faults and strictly managing memory boundaries.

5.4 Reliability Requirements

The application must execute with zero memory leaks during dynamic memory allocation and cleanup.

The cache eviction policy must reliably retain the most relevant "hot" data under heavy workloads.

5.5 Scalability Requirements

The storage engine's latency must scale independently of the overall dataset size.

The system must maintain constant performance whether processing 1,000, 100,000, or 1,000,000 continuous requests.

5.6 Maintainability Requirements

The codebase must feature a modular architecture, clearly separating the storage engine API from the internal linked-list pointer logic.

Code must be written in a systems-level language like C++, Go, or Rust to allow for straightforward compilation using standard build tools like CMake or Makefiles.

6. Other Requirements

Formal mathematical analysis using Big-O notation must be generated to mathematically justify why the engine's operations occur in constant time, alongside calculating the specific memory byte overhead required per node.

Appendix A: Glossary

DLL: *Doubly Linked List, a structure utilizing prev and next pointers to achieve $O(1)$ positional operations.*

LRU: *Least Recently Used, a cache replacement algorithm that targets the oldest, unaccessed data for eviction.*

$O(1)$: *Constant time complexity, indicating execution speed remains fixed regardless of n (the dataset size)*

Appendix B: Analysis Models

Data Structure Layout Diagram: Visualizes an array representing Hash Map buckets with pointers mapping directly into specific nodes of a Doubly Linked List, noting the Head and Tail nodes .

Operational Flowcharts: Logic tracing for get() and put() paths, defining hit checks, capacity bounds, and internal pointer rewiring logic.

Empirical Benchmarking Plots: * O(1) Time Scalability Graph (Operations vs Execution Time per Operation).

- Benchmark vs Baseline Comparison Graph.
- Hit-Rate Efficiency Graph under synthetic workloads

Appendix C: To Be Determined List

TBD 1: Future architectural extensions to adapt the engine for multi-threading thread safety constraints.

TBD 2: Optional future application of persistence or Write-Ahead Logging integration.

